eProceeding of TIK
E-ISSN: 2797-1724, P-ISSN: 3031-1454
Vol. 7 No. 1 2027 pp. xx-xx

Replikasi *Master-Slave* dan *Backup* Otomatis untuk Pemeriksaan Rawat Inap Berbasis Jaringan *Hybrid*

Al Nazirul Mubin[1], Hari Toha Hidayat[2*], Muhammad Davi[3]

[1,2,3] Jurusan Teknologi Informasi dan Komputer Politeknik Negeri Lhokseumawe, Indonesia

*Penulis Korespondensi: haritoha@pnl.ac.id

This is an open access article under the CC BY-SA license.

Abstrak

Ketersediaan dan keutuhan data rekam medis pasien rawat inap merupakan aspek kritikal dalam operasional sistem informasi manajemen rumah sakit (SIMRS), yang sering terganggu oleh efek penguncian tabel (*table locking*) saat pencadangan konvensional berjalan bersamaan dengan operasional sistem. Permasalahan yang diangkat adalah sejauh mana arsitektur replikasi *master-slave* terintegrasi *cron job* dan mariadb-*backup* efektif mencegah *table locking*, serta sejauh mana efisiensi komputasi dan resiliensi sinkronisasi yang dicapai berdasarkan pengukuran durasi pencadangan dan latensi replikasi. Penelitian ini bertujuan membuktikan efektivitas arsitektur tersebut dalam mencegah *table locking* dan mencapai layanan tanpa jeda (*zero-downtime*), sekaligus mengukur efisiensi dan resiliensi sinkronisasi sistem pada kondisi normal maupun beban tinggi. Metode yang digunakan adalah kuantitatif eksperimental, mengimplementasikan *hot backup* melalui topologi replikasi *master-slave* MariaDB terintegrasi *cron job* pada jaringan *hybrid*, dievaluasi menggunakan metode *average time* dan instrumen *pt-heartbeat*. Hasil pengujian menunjukkan durasi rata-rata *full backup* 9,1 detik dan *incremental backup* 8,7 detik, setara efisiensi 4,39%, dengan latensi stabil pada 54–55,5 ms. Ditemukan pula bahwa konflik *primary key* (*error* 1062) dapat menghentikan *SQL thread* replikasi, sehingga tetap diperlukan pemantauan aktif untuk mencegah data usang (*stale data*) pada berkas cadangan.

Kata kunci: *average time*, *backup* otomatis, jaringan *hybrid*, *master-slave*, replikasi basis data, rawat inap

1. Pendahuluan

Adopsi jaringan *hybrid* yang menggabungkan infrastruktur lokal dan *cloud* menjadi ciri modernisasi teknologi informasi di era Industri 4.0, karena mampu meningkatkan efisiensi infrastruktur sekaligus mendorong transformasi jaringan organisasi [1]. Arsitektur ini menuntut ketersediaan data selama 24 jam, sementara kompleksitas sinkronisasi antarinfrastruktur meningkatkan risiko kehilangan data akibat kegagalan perangkat keras, gangguan konektivitas, maupun ancaman siber.

Sektor kesehatan merasakan tekanan ini secara langsung. Rumah sakit yang beralih ke sistem informasi manajemen rumah sakit (SIMRS) terintegrasi menghasilkan data klinis rawat inap secara terus-menerus dan dalam volume besar [2], termasuk dalam pengelolaan rekam medis elektronik yang menuntut ketersediaan data tinggi [3], sementara institusi kesehatan menghadapi risiko digital yang lebih tinggi dibandingkan sektor lain karena karakteristik datanya yang sensitif dan kritis [4]. Ancaman keamanan pada SIMRS turut diperparah oleh keterbatasan infrastruktur pendukung ketersediaan data [5], dan kasus kebocoran data kesehatan skala nasional seperti insiden BPJS Kesehatan menjadi pengingat nyata atas risiko tersebut [6]. Pada unit rawat inap, *downtime* atau hilangnya data pasien bukan sekadar gangguan administratif, melainkan potensi ancaman terhadap kontinuitas perawatan. Replikasi *master-slave* menjadi solusi yang relevan karena memungkinkan penggandaan data secara *real-time* dari server utama ke server cadangan, sehingga meminimalkan risiko *single point of failure* [7]. Namun replikasi saja belum cukup; pencadangan yang berjalan bersamaan dengan operasional sistem berisiko menimbulkan *table locking* jika tidak ditangani dengan metode *hot backup* yang tepat [8].

Beberapa penelitian terdahulu telah menyentuh area ini dari sudut berbeda. Sholihah dan Darujati mengimplementasikan replikasi MySQL *master-slave* di dalam *Docker Container* [9]. Kamiswara dan Wijaya menunjukkan bahwa pemisahan operasi baca-tulis pada replikasi *master-slave* MariaDB menurunkan penggunaan CPU secara signifikan [10]. Susilo dkk. mengembangkan SIMRS berbasis *framework* PHP untuk kebutuhan operasional rumah sakit [11], sementara penelitian lain melakukan sosialisasi konsep replikasi *database* kepada staf IT rumah sakit [12]. Haryoto dkk. mengevaluasi performa MySQL *master-slave* di lingkungan *cloud* dan berhasil meningkatkan *response time* pada volume data besar [13], sejalan dengan penelitian ketersediaan basis data tinggi pada infrastruktur *cloud* lain [14]. Hulu dkk. menerapkan replikasi MySQL pada CMS WordPress untuk mengurangi risiko kehilangan data [15]. Hidayat dkk. mengembangkan *chatbot* ketersediaan ruang rawat inap terintegrasi SIMRS [16].

Dari studi-studi tersebut terlihat pola yang cukup konsisten: penelitian teknis tentang replikasi *master-slave* umumnya tidak dilakukan pada domain kesehatan, sedangkan penelitian di domain kesehatan cenderung berhenti pada sosialisasi atau antarmuka aplikasi. Celah dan posisi kebaruan penelitian ini dibahas lebih lanjut pada bagian Metode.

Penelitian ini mengusulkan arsitektur replikasi *master-slave* MySQL yang diintegrasikan dengan mariadb-*backup* untuk eksekusi *hot backup* yang sejalan dengan pendekatan otomatisasi *backup* berbasis *cron job* pada infrastruktur *cloud* yang telah dibuktikan efektif pada konteks lain [17] dan *cron job* untuk penjadwalan otomatis, diterapkan pada jaringan *hybrid* sistem pemeriksaan pasien rawat inap. Pendekatan ini juga sejalan dengan prinsip pemulihan bencana (*disaster recovery*) pada sistem *cloud* untuk meminimalkan kehilangan data dan waktu henti layanan [18]. Efektivitasnya diukur melalui durasi eksekusi pencadangan menggunakan metode *average time* dan stabilitas latensi replikasi menggunakan instrumen *pt-heartbeat*, pada kondisi operasional normal maupun saat terjadi lonjakan beban kerja.

2. Metode

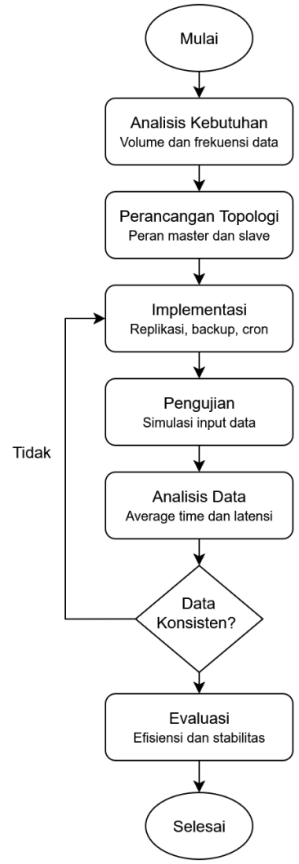
2.1. *Gap* dan *Novelty* Penelitian

Kajian atas berbagai skema replikasi *database* menunjukkan bahwa sebagian besar solusi yang tersedia, baik *open source* maupun komersial, dirancang untuk kebutuhan umum dan belum menyasar karakteristik data kesehatan yang sensitif dan bersifat *real-time critical* [19]. Perbandingan dengan penelitian terdahulu menunjukkan beberapa celah yang relevan dengan penelitian ini. Sholihah dan Darujati berfokus pada isolasi lingkungan menggunakan *Docker Container*, tanpa menyentuh penggunaan VPS *native* dengan penjadwalan *incremental backup* otomatis yang krusial untuk skalabilitas basis data rumah sakit [9]. Kamiswara dan Wijaya berhasil mengoptimalkan performa kueri melalui pemisahan operasi baca-tulis, tetapi belum menyediakan mekanisme perlindungan data otomatis untuk skenario pemulihan bencana pada infrastruktur VPS *cloud* [10]. Penelitian ketersediaan tinggi pada basis data *open source* di lingkungan *cloud* juga masih berfokus pada pemantauan performa, bukan otomatisasi pencadangan fisik [14]. Haryoto dkk. berfokus pada *benchmarking* performa sistem, tanpa membahas prosedur *backup* otomatis yang menjamin operasional tetap berjalan tanpa risiko *table locking* [13]. Hulu dkk. menggunakan *plugin* pada CMS WordPress, sehingga belum menjawab kebutuhan sistem informasi kesehatan murni yang memerlukan kontrol di level server [15]. Hidayat dkk. relevan dari sisi konteks rumah sakit, tetapi berfokus pada antarmuka pengguna, sehingga menyisakan celah pada strategi redundansi data di sisi *backend* [16]. Kesenjangan antara solusi replikasi tingkat akademik dan kebutuhan operasional nyata ini turut sejalan dengan temuan bahwa implementasi *middleware* replikasi *database* seringkali belum menjembatani kebutuhan performa, ketersediaan, dan administrasi secara bersamaan pada sistem produksi [20].

Berdasarkan celah tersebut, penelitian ini menawarkan beberapa unsur kebaruan. Pertama, penggunaan mariadb-*backup* yang dipadukan dengan *cron job* untuk otomatisasi *backup* pada replikasi *master-slave* MySQL di lingkungan VPS belum banyak dibahas dalam literatur yang ada. Kedua, karakteristik data sistem pemeriksaan pasien rawat inap yang bersifat *real-time critical*, berbeda dari kasus akademik maupun *e-commerce* yang umum dijadikan objek penelitian sejenis. Ketiga, integrasi dengan jaringan *hybrid* menambah kompleksitas sekaligus relevansi praktis dibanding pengujian pada jaringan tunggal. Keempat, penjadwalan *cron job* untuk *backup* terjadwal menjadi fokus yang berbeda dari isu *failover* atau optimasi performa yang lebih dominan pada penelitian sebelumnya. Kelima, pengujian dilakukan langsung pada infrastruktur VPS *production-grade*, bukan pada lingkungan virtualisasi lokal.

2.2. Desain Penelitian

Penelitian ini menggunakan pendekatan kuantitatif eksperimental untuk menguji keandalan sistem otomatisasi *backup* pada jaringan *hybrid* sistem pemeriksaan pasien rawat inap. Pendekatan ini dipilih karena efektivitas sistem perlu dibuktikan melalui data terukur, bukan sekadar observasi kualitatif atas hasil implementasi. Gambar 1 menunjukkan alur pelaksanaan penelitian, mulai dari analisis kebutuhan hingga evaluasi hasil, dengan satu titik keputusan pada tahap analisis data untuk memastikan konsistensi sebelum penelitian dinyatakan selesai. Keputusan ini menjadi mekanisme kontrol kualitas internal: apabila data hasil pengujian pada *node master* dan *slave* belum sepenuhnya konsisten, konfigurasi pada tahap implementasi ditinjau ulang sebelum proses berlanjut, sehingga hasil akhir yang dievaluasi benar-benar mencerminkan kondisi sistem yang sudah stabil. Rincian tiap tahap, mulai dari kebutuhan data hingga rancangan perangkat sistem, dijelaskan pada subbab berikut.



Gambar 1. *Flowchart* Desain Penelitian

Tahapannya mencakup analisis kebutuhan, perancangan topologi, hingga implementasi replikasi *master-slave* pada lingkungan *Virtual Private Server* (VPS). Setelah pengujian, data dianalisis untuk memastikan konsistensi antara *node master* dan *slave*; apabila ditemukan ketidaksesuaian, konfigurasi implementasi ditinjau ulang sebelum proses dilanjutkan ke tahap evaluasi. Variabel yang diukur meliputi efisiensi durasi *backup* dan stabilitas sinkronisasi data, keduanya dianalisis untuk memastikan konsistensi dan ketersediaan informasi medis secara berkelanjutan.

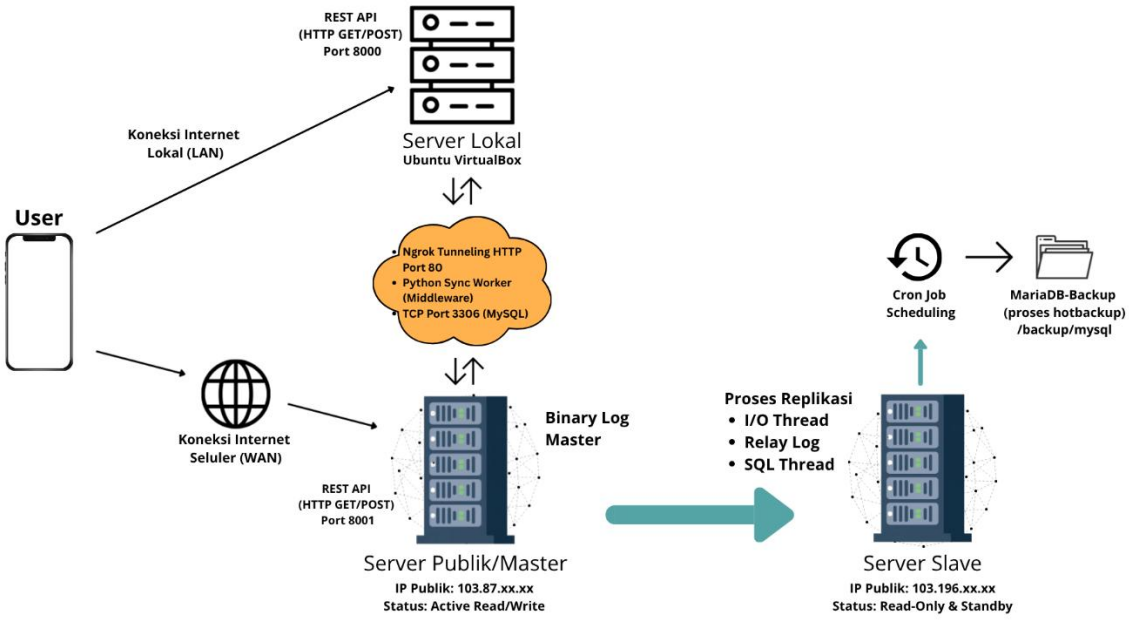
2.3. Rancangan Sistem

Sistem dirancang untuk beroperasi pada jaringan *hybrid* yang menghubungkan terminal pemeriksaan pasien rawat inap dengan pusat data. Dari sisi perangkat keras, arsitektur terdiri atas server *master* yang menangani transaksi pasien secara *real-time*, server *slave* yang berfungsi sebagai replika data sekaligus lokasi eksekusi *backup*, dua unit VPS sebagai infrastruktur jaringan yang diamankan melalui pendekatan keamanan berlapis [21], dan terminal *client* di unit rawat inap untuk *input* data. Dari sisi perangkat lunak, sistem berjalan di atas Linux Ubuntu Server dengan MySQL sebagai *database management system* yang mengaktifkan fitur replikasi, mariadb-*backup* sebagai mesin *hot backup*, dan *cron job* sebagai alat penjadwalan otomatis. Berbeda dari metode ekspor logis konvensional seperti *mysqldump*, mariadb-*backup* mencapai *hot backup* dengan menyalin berkas data fisik langsung dari level sistem operasi tanpa menghentikan proses tulis pada *slave*, sehingga replikasi tetap berjalan selama pencadangan berlangsung.

Rancangan *input* sistem mencakup data identitas pasien, rekam medis, dan hasil pemeriksaan yang dimasukkan melalui aplikasi rawat inap; parameter koneksi antara server *master* dan *slave* (alamat IP, kredensial, dan *log-bin file*); serta jadwal spesifik (pukul 02:00 WIB) untuk memulai proses *backup* otomatis. Pada tahap proses, setiap data yang masuk ke *master* dicatat dalam *binary log* dan dikirim ke *slave* untuk menjaga konsistensi, kueri tulis diarahkan ke *master* sementara kueri baca diarahkan ke *slave*, dan *cron job* menjalankan skrip yang memanggil mariadb-*backup* tanpa mengunci tabel (*non-blocking*) sesuai jadwal. Keluaran sistem berupa basis data pada *slave* yang identik dengan *master* dan siap digunakan saat *failover*, berkas cadangan terkompresi yang tersimpan di direktori cadangan, serta data numerik durasi setiap siklus *backup* sebagai bahan analisis performa.

2.4. Arsitektur Sistem

Gambar 2 menunjukkan alur sistem yang mengintegrasikan jaringan *hybrid* dengan topologi replikasi *master-slave*, mulai dari jalur transmisi data pasien hingga proses pencadangan otomatis di *node slave*.



Gambar 2. Alur Sistem Jaringan *Hybrid* dan *Master-Slave*

Alur operasional sistem bekerja sebagai berikut. Pengguna mengakses aplikasi melalui jalur lokal (LAN, *port* 8000) atau publik (WAN, *port* 8001), yang terhubung ke jaringan *hybrid* melalui *Ngrok Tunneling* dan *Python Sync Worker* untuk sinkronisasi basis data pada *port* 3306. Transaksi baca dan tulis diproses secara *real-time* pada *node master* melalui *binary log*, kemudian direplikasi secara otomatis ke *node slave* melalui mekanisme *I/O thread*, *relay log*, dan *SQL thread*. Setiap perubahan data pada *master* baik penyisipan, pembaruan, maupun penghapusan akan dicatat sebagai peristiwa (*event*) dalam *binary log*, pendekatan yang juga terbukti efektif membangun sinkronisasi data antarhost pada studi lain [22]; *I/O thread* pada *slave* kemudian menyalin peristiwa tersebut ke *relay log*, dan *SQL thread* mengeksekusi ulang perintah yang sama secara berurutan pada *slave*. Mekanisme eksekusi ulang inilah yang menjaga data di *slave* tetap identik dengan *master*, bukan sekadar hasil salin berkas mentah. Proses *hot backup* diisolasi sepenuhnya pada *node slave* menggunakan *cron job* dan mariadb-*backup*, menghasilkan berkas cadangan otomatis di direktori */backup/mysql*.

2.5. Tahapan Penelitian

Penelitian ini dilaksanakan melalui lima tahap. Tahap analisis kebutuhan mengidentifikasi volume dan frekuensi penulisan data dari jaringan *hybrid* untuk menentukan jadwal *cron job* yang optimal. Tahap perancangan topologi menetapkan pembagian peran server: *node master* menangani transaksi pasien secara *real-time*, sedangkan *node slave* didedikasikan untuk replikasi data dan proses *backup*, sehingga sumber daya *master* tidak terbebani. Tahap implementasi mencakup konfigurasi replikasi melalui *binary logging*, integrasi mariadb-*backup* untuk pencadangan tanpa mengunci tabel (*non-locking*), serta penjadwalan otomatis melalui *cron job*. Tahap pengujian dilakukan dengan mensimulasikan *input* data pasien pada jaringan *hybrid*, kemudian memantau status replikasi untuk memastikan penyalinan data ke *node slave* berjalan tanpa hambatan. Tahap evaluasi mengumpulkan data temporal dari setiap siklus *backup* untuk dianalisis dengan metode *average time*, guna mengukur konsistensi sistem dalam menyinkronkan data medis di tengah fluktuasi beban kerja.

Tabel 1. Tahapan Penelitian

Tahap	Fokus Kegiatan
Analisis kebutuhan	Identifikasi volume dan frekuensi data untuk jadwal *cron job*
Perancangan topologi	Pembagian peran *node master* dan *node slave*
Implementasi	Konfigurasi replikasi, mariadb-*backup*, *cron job*
Pengujian	Simulasi *input* data dan pemantauan status replikasi
Evaluasi	Analisis durasi *backup* dengan metode *average time*

2.6. Metode Pengukuran

Efektivitas sistem diukur melalui dua metrik. Pertama, *average time* digunakan untuk menghitung rata-rata durasi setiap siklus *backup* otomatis, dengan formula:

$$Average\ Time = \frac{\sum_{i=1}^{n} Durasi\ Backup_i}{n} \quad (1)$$

Dengan n adalah jumlah sesi *backup* yang diuji dan $Durasi\ Backup_i$ adalah waktu yang dicatat pada sesi *backup* ke-i dari *log* sistem. Nilai rata-rata dipilih karena mampu meredam anomali dari lonjakan beban CPU atau *delay* jaringan yang bersifat sesaat, sehingga hasil pengukuran lebih stabil untuk dibandingkan antarkonfigurasi [23].

Kedua, latensi replikasi diukur menggunakan instrumen *pt-heartbeat* pada *node master* dengan interval 0,1 detik, guna memperoleh presisi milidetik yang tidak dapat dicapai oleh parameter bawaan MySQL *Seconds_Behind_Master* yang hanya terbatas pada satuan detik. Pada *node slave*, skrip otomatis menghitung selisih *timestamp* dan mengekspor hasilnya ke format CSV untuk diolah lebih lanjut menggunakan rumus *average time* yang sama. Pengujian dilakukan pada dua skenario: kondisi normal (*idle*) dan kondisi beban tinggi saat proses *backup* berlangsung.

2.7. Skenario Pengujian

Pengujian dilakukan melalui dua skenario. Skenario pertama merekam latensi replikasi selama tiga menit pada kondisi normal (*idle*), yaitu saat jaringan *hybrid* tidak sedang menerima aktivitas *input* data rekam medis dalam volume besar. Skenario kedua merekam latensi pada kondisi beban tinggi, yaitu saat proses *full backup* sedang berjalan bersamaan dengan transaksi data pasien. Kedua skenario menghasilkan data latensi yang selanjutnya diolah menggunakan metode *average time* untuk dibandingkan.

3. Hasil dan Pembahasan

3.1. Konsistensi Data pada Replikasi

Pengujian konsistensi dilakukan dengan menginjeksi 50 baris data pemeriksaan pasien rawat inap ke basis data utama pada *node master*, kemudian dilanjutkan dengan verifikasi silang (*cross-validation*) pada kedua server. Verifikasi mencakup dua hal: memastikan parameter *Slave_IO_Running* dan *Slave_SQL_Running* tetap berstatus *Yes* selama transaksi berlangsung, serta membandingkan hasil kueri SELECT COUNT(*) dan kesamaan nilai *primary key* pada tabel-tabel kritis antara *node master* dan *node slave*. Gambar 3 dan Gambar 4 menunjukkan hasil kueri validasi pada *node master* dan *node slave* secara berdampingan.

```
MariaDB [rawat_inap_vps]> SELECT COUNT(*) FROM pasien;
+----------+
| COUNT(*) |
+----------+
|       50 |
+----------+
1 row in set (0.002 sec)

MariaDB [rawat_inap_vps]> SELECT COUNT(*) FROM users;
+----------+
| COUNT(*) |
+----------+
|        6 |
+----------+
1 row in set (0.001 sec)

MariaDB [rawat_inap_vps]> SELECT COUNT(*) FROM visit;
+----------+
| COUNT(*) |
+----------+
|       50 |
+----------+
1 row in set (0.001 sec)
```

Gambar 3. Kueri Validasi Baris Data pada *Node Master*

```
MariaDB [rawat_inap_vps]> SELECT COUNT(*) FROM pasien;
+----------+
| COUNT(*) |
+----------+
|       50 |
+----------+
1 row in set (0.051 sec)

MariaDB [rawat_inap_vps]> SELECT COUNT(*) FROM users;
+----------+
| COUNT(*) |
+----------+
|        6 |
+----------+
1 row in set (0.006 sec)

MariaDB [rawat_inap_vps]> SELECT COUNT(*) FROM visit;
+----------+
| COUNT(*) |
+----------+
|       50 |
+----------+
1 row in set (0.001 sec)
```

Gambar 4. Kueri Validasi Baris Data pada *Node Slave*

Tabel 2. Komparasi Jumlah Baris dan Konsistensi Objek Basis Data

No	Nama Tabel	Jumlah Baris (*Master*)	Jumlah Baris (*Slave*)	Selisih (*Delta*)	Status
1	pasien	50	50	0	Konsisten
2	users	6	6	0	Konsisten
3	visit	50	50	0	Konsisten

Kedua gambar tersebut memperlihatkan bahwa hasil kueri SELECT COUNT(*) pada kedua server menunjukkan angka yang identik untuk setiap tabel, sejalan dengan data pada Tabel 2. Ketiga baris tabel menunjukkan selisih (*delta*) sebesar 0 pada seluruh baris yang diamati. Hasil ini mengonfirmasi bahwa setiap transaksi data rekam medis yang dituliskan pada *node master* diteruskan oleh *binary log* dan dieksekusi ulang secara utuh oleh *SQL thread* pada *node slave*, tanpa ada baris yang hilang atau tertinggal selama proses replikasi.

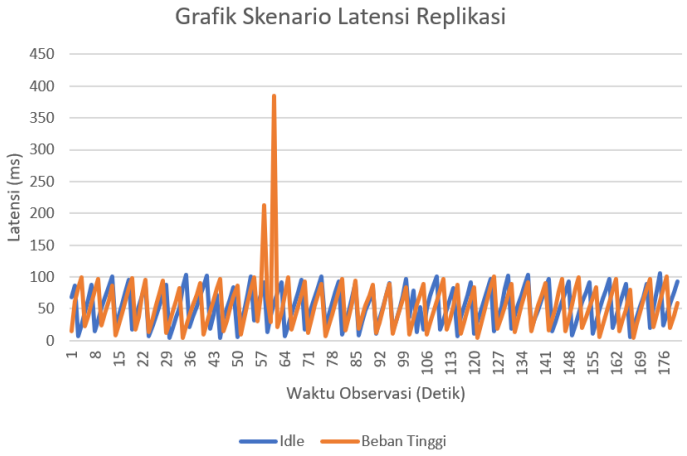
3.2. Latensi Replikasi

Latensi replikasi diukur pada dua kondisi: normal (*idle*) dan beban tinggi saat proses *full backup* berlangsung di *node slave*. Setiap skenario direkam selama tiga menit dengan interval 0,1 detik, menghasilkan 180 sampel per skenario.

Tabel 3. Komparasi Latensi Replikasi pada Dua Skenario Pengujian

Parameter	Kondisi Normal (*Idle*)	Kondisi Beban Tinggi (*Full Backup*)
Latensi minimum	4,3 ms	3,9 ms
Latensi maksimum	105,3 ms	385,3 ms
Rata-rata latensi	54 ms	55,5 ms

Rata-rata latensi hanya naik dari 54 ms pada kondisi normal menjadi 55,5 ms saat proses *backup* berjalan bersamaan dengan replikasi, jauh di bawah ambang batas toleransi 1.000 ms. Gambar 5 menampilkan pola fluktuasi latensi dari kedua skenario berdasarkan data instrumen *pt-heartbeat* selama tiga menit observasi.



Gambar 5. Komparasi Fluktuasi Latensi Replikasi pada Kondisi Normal dan Beban Tinggi

Lonjakan latensi maksimum hingga 385,3 ms pada kondisi beban tinggi terlihat sebagai puncak sesaat pada grafik, mengindikasikan kontensi sumber daya *disk I/O* di *node slave* yang memicu antrean singkat pada *SQL thread*. Grafik kemudian kembali stabil dalam hitungan detik, menunjukkan bahwa mekanisme *catch-up* pada replikasi MySQL mampu mengisolasi beban pencadangan di server cadangan sehingga operasional *input* data pada *node master* tetap berjalan *real-time* tanpa risiko *table locking*.

3.3. Efisiensi Durasi *Backup*

Durasi otomatisasi *backup* dianalisis pada dua kategori sesuai jadwal *cron job*: *full backup* harian pukul 02:00 WIB dan *incremental backup* pada pukul 09:00, 15:00, dan 21:00 WIB. *Full backup* menyalin seluruh berkas basis data secara utuh, sementara *incremental backup* hanya menyalin data yang berubah sejak pencadangan terakhir. Gambar 6 menunjukkan isi direktori hasil *backup* pada salah satu tanggal pengujian, sebagai bukti bahwa *cron job* benar-benar mengeksekusi dan menghasilkan berkas cadangan sesuai jadwal yang ditentukan.

```
[redacted] ls -lh /backup/mysql/2026-06-14
total 16K
drwxr-xr-x 9 root root 4.0K Jun 14 02:00 full
drwxr-xr-x 9 root root 4.0K Jun 14 09:00 inc-09
drwxr-xr-x 9 root root 4.0K Jun 14 15:00 inc-15
drwxr-xr-x 9 root root 4.0K Jun 14 21:00 inc-21
[redacted] ls -lh /backup/mysql/2026-06-14/full
total 125M
-rw-r----- 1 root root 416K Jun 14 02:00 aria_log.00000001
-rw-r----- 1 root root   52 Jun 14 02:00 aria_log_control
-rw-r----- 1 root root  285 Jun 14 02:00 backup-my.cnf
-rw-r----- 1 root root  49M Jun 14 02:00 ib_logfile0
-rw-r----- 1 root root  76M Jun 14 02:00 ibdata1
drwx------ 2 root root 4.0K Jun 14 02:00 monitor
drwx------ 2 root root 4.0K Jun 14 02:00 mysql
drwx------ 2 root root 4.0K Jun 14 02:00 performance_schema
drwx------ 2 root root 4.0K Jun 14 02:00 phpmyadmin
drwx------ 2 root root 4.0K Jun 14 02:00 politeknik
drwx------ 2 root root 4.0K Jun 14 02:00 rawat_inap_vps
drwx------ 2 root root  12K Jun 14 02:00 sys
-rw-r----- 1 root root   35 Jun 14 02:00 xtrabackup_binlog_info
-rw-r----- 1 root root  107 Jun 14 02:00 xtrabackup_checkpoints
-rw-r----- 1 root root  543 Jun 14 02:00 xtrabackup_info
```

Gambar 6. Isi Direktori Hasil *Backup* pada *Node Slave*

Tabel 4. Durasi Rata-Rata Otomatisasi *Backup*

Jenis *Backup*	Jumlah Sesi (n)	Durasi Rata-rata
Full backup	7	9,1
Incremental backup	9	8,7

Berdasarkan perbandingan kedua durasi tersebut, tingkat efisiensi waktu dihitung dengan:

$$Tingkat\ Efisiensi = \frac{T_{full} - T_{incremental}}{T_{full}} \text{ X } 100\% = \frac{9{,}1 - 8{,}7}{9{,}1} \text{ X } 100\% = 4{,}39\% \quad (2)$$

Penerapan *incremental backup* pada jam sibuk pelayanan rawat inap menghasilkan efisiensi waktu eksekusi sebesar 4,39% dibandingkan *full backup*. Selisih ini memang berada pada skala detik, tetapi tetap relevan mengingat volume data medis yang terus bertambah, karena durasi eksekusi yang singkat baik untuk *full* maupun *incremental backup* membantu mengurangi beban *disk I/O* dan memori server selama masa pencadangan berlangsung.

3.4. Ketahanan Sistem: Konflik Sinkronisasi dan Failover

Dua pengujian dilakukan untuk memetakan batas ketahanan sistem: simulasi konflik data pada replikasi, dan simulasi kegagalan *node master* (*failover*).

Konflik sinkronisasi disimulasikan dengan menginjeksi data pasien ilegal langsung ke *node slave* (ID: 90), lalu menginput data sah dengan *primary key* yang sama melalui aplikasi SIMRS yang terhubung ke *node master*. Injeksi ganda ini memicu benturan identitas (*duplicate key*) saat *relay log* pada *slave* mencoba menerapkan transaksi dari *master*. Gambar 7 menunjukkan *log error* yang tercatat pada status replikasi saat konflik ini terjadi.

```
             Slave_IO_Running: Yes
            Slave_SQL_Running: No
              Last_SQL_Error: Error 'Duplicate entry '90' for key 'PRIMARY'' on query. Default database: 'rawat_inap_vps'.
 Query: 'INSERT INTO `pasien` (`id`, `no_rm`, `nama`, `jenis_kelamin`, `tanggal_lahir`, `created_at`, `updated_at`, `dokter_
id`, `dokter_uuid`, `is_active`, `uuid`, `status_sync`, `synced_at`, `source_server`, `status`, `tanggal_keluar`, `catatan_k
eluar`, `action_type`, `is_deleted`) VALUES ('90', 'RM00055', 'Abdul Kodir', 'L', '2003-04-08', '2026-06-22 23:39:53', '2026
-06-22 23:39:53', NULL, NULL, '1', NULL, 'pending', NULL, 'vps', NULL, NULL, NULL, 'create', '0')'
```

Gambar 7. *Log Error Duplicate Entry* pada Status Replikasi *Node Slave*

Tabel 5. Evaluasi Skenario Kegagalan Sinkronisasi

Tahap	Kondisi	Slave_IO_Running	Slave_SQL_Running	Keterangan
1	Injeksi data ilegal ke *slave*	*Yes*	*Yes*	Replikasi berjalan normal
2	Injeksi data sah ke *master* dengan ID sama	*Yes*	*No*	*Error 1062: Duplicate entry* menghentikan *SQL thread*
3	Pemulihan oleh administrator	*Yes*	*Yes*	Data ilegal dihapus manual, sistem *catch-up*

Pada tahap 2, *Slave_IO_Running* tetap berstatus *Yes*, tetapi *Slave_SQL_Running* berubah menjadi *No* akibat *error 1062*, sesuai dengan yang tercatat pada Gambar 6. Temuan ini mengungkap celah arsitektural yang perlu diwaspadai: kegagalan replikasi yang tidak segera ditangani membuat *cron job* tetap berjalan sesuai jadwal dan menghasilkan berkas cadangan berisi data usang (*stale data*), karena log terbaru tertahan di *relay log* dan tidak diketahui oleh proses *backup*. Pemulihan memerlukan intervensi manual administrator melalui tiga perintah berurutan: menghentikan replikasi sementara (STOP SLAVE;), menghapus baris data penyebab konflik (DELETE FROM pasien WHERE id = 90;), dan mengaktifkan kembali replikasi (START SLAVE;).

Pengujian kedua mensimulasikan kegagalan *node master* dengan menghentikan paksa layanan MySQL (systemctl stop mariadb), untuk menguji apakah *node slave* dapat mengambil alih peran melayani kueri baca. Gambar 8 dan Gambar 9 menunjukkan status *slave* masing-masing saat *node master* aktif dan saat *node master* tidak aktif.

```
MariaDB [(none)]> SHOW SLAVE STATUS\G;
*************************** 1. row ***************************
               Slave_IO_State: Waiting for master to send event
                  Master_Host: [redacted]
                  Master_User: [redacted]
                  Master_Port: 3306
                Connect_Retry: 60
              Master_Log_File: mysql-bin.000007
          Read_Master_Log_Pos: 342
               Relay_Log_File: mysqld-relay-bin.000041
                Relay_Log_Pos: 641
        Relay_Master_Log_File: mysql-bin.000007
             Slave_IO_Running: Yes
            Slave_SQL_Running: Yes
              Replicate_Do_DB:
```

Gambar 8. Status *Slave* Ketika *Node Master* Aktif

```
MariaDB [(none)]> SHOW SLAVE STATUS\G;
*************************** 1. row ***************************
               Slave_IO_State: Reconnecting after a failed master event read
                  Master_Host: [redacted]
                  Master_User: [redacted]
                  Master_Port: 3306
                Connect_Retry: 60
              Master_Log_File: mysql-bin.000006
          Read_Master_Log_Pos: 312412
               Relay_Log_File: mysqld-relay-bin.000039
                Relay_Log_Pos: 312711
        Relay_Master_Log_File: mysql-bin.000006
             Slave_IO_Running: Connecting
            Slave_SQL_Running: Yes
              Replicate_Do_DB:
```

Gambar 9. Status *Slave* Ketika *Node Master Down*

Tabel 6. Hasil Pengujian Skenario Ketersediaan (*Failover*)

Tahap	Kondisi *Master*	Aktivitas Akses	Hasil
1	*Active*	Tulis dan baca	Data tersimpan dan tersinkronisasi normal
2	*Down*	Tulis baru	Kueri gagal, koneksi ke *master* terputus
3	*Down*	Baca data lama	Aplikasi tetap menampilkan rekam medis dari *node slave*

Kedua gambar tersebut memperkuat data pada Tabel 6: saat *node master* tidak aktif, transaksi penulisan data baru berhenti sementara, tetapi pemanggilan rekam medis historis tetap berjalan karena sistem mengalihkan koneksi baca ke *node slave*. Pengalihan ini memungkinkan tenaga medis tetap mengakses data pasien terakhir yang tersinkronisasi meski *node master* sedang pulih.

Kedua pengujian ini memperlihatkan gambaran yang lebih lengkap dibanding sekadar mengonfirmasi keberhasilan sistem. Mekanisme *failover* terbukti efektif menjaga ketersediaan layanan baca, tetapi keandalan itu bergantung pada satu syarat: replikasi harus dalam kondisi sehat saat kegagalan terjadi. Skenario konflik *primary key* menunjukkan bahwa begitu *SQL thread* berhenti akibat *error 1062*, seluruh siklus di baliknya turut terganggu, *node slave* yang seharusnya jadi sumber data cadangan yang andal justru berisiko menyimpan versi data yang sudah usang tanpa peringatan otomatis. Artinya, otomatisasi *backup* dan replikasi *master-slave* pada penelitian ini efektif untuk kondisi operasional normal, tetapi tetap memerlukan lapisan pemantauan aktif atas status *thread* replikasi agar potensi *stale data* dapat terdeteksi sebelum memengaruhi keandalan cadangan yang dihasilkan.

4. Kesimpulan

Arsitektur replikasi *master-slave* MySQL yang diintegrasikan dengan mariadb-*backup* dan *cron job* terbukti efektif menjalankan otomatisasi *hot backup* tanpa menimbulkan *table locking*. Beban kerja pencadangan sepenuhnya terisolasi di *node slave*, sehingga layanan *input* rekam medis pada *node master* tetap berjalan tanpa jeda operasional (*zero-downtime*). Meski demikian, sistem ini tetap memerlukan pengawasan administrator, mengingat

konflik sinkronisasi (*error* 1062) dapat memicu pembentukan data usang (*stale data*) pada berkas cadangan apabila tidak segera ditangani.

Dari sisi efisiensi, pengukuran metode *average time* mencatat durasi rata-rata *full backup* sebesar 9,1 detik dan *incremental backup* sebesar 8,7 detik, setara efisiensi waktu pemrosesan sebesar 4,39%. Pemantauan *pt-heartbeat* juga menunjukkan latensi replikasi yang stabil, dengan rata-rata 54 ms pada kondisi normal dan 55,5 ms saat beban tinggi, jauh di bawah ambang batas toleransi kritis. Kedua temuan ini menegaskan bahwa sistem yang dibangun cukup efisien dan tangguh untuk lingkungan produksi rawat inap, dengan catatan bahwa keandalannya tetap bergantung pada pemantauan aktif terhadap status replikasi.

Untuk pengembangan lebih lanjut, penelitian berikutnya disarankan mengintegrasikan dasbor pemantauan berbasis web, misalnya melalui Zabbix dengan notifikasi otomatis sebagaimana telah diterapkan pada konteks jaringan lain [24], beserta otomatisasi administrasi tambahan melalui skrip *shell* dan *cron job* yang telah terbukti efektif pada studi sejenis [25], guna memvisualisasikan latensi *pt-heartbeat* dan status replikasi secara *real-time* tanpa bergantung pada *command line interface*. Langkah ini dapat mempercepat deteksi anomali sinkronisasi sebelum berdampak pada integritas data cadangan.

REFERENSI

[1] A. Z. Nabila, S. Yasmine, dan A. Stevano, "Peran Cloud Computing terhadap Efisiensi Infrastruktur dan Transformasi Jaringan Organisasi: Studi Literatur," *Jurnal Sains dan Teknologi 4.0 (JST 4.0)*, vol. 3, no. 1, Des 2025, Diakses: 15 Agustus 2026. [Daring]. Tersedia pada: https://jurnal.komputasi.org/index.php/jst/article/download/40/33/77

[2] P. Maulaningrum, S. Mujanah, dan A. Y. A. Fianto, "Transformasi Digital di Sektor Kesehatan Tinjauan Literatur tentang Penerapan Teknologi Informasi dalam Manajemen Pelayanan," *Jurnal Ilmu Manajemen, Ekonomi dan Kewirausahaan*, vol. 5, no. 1, hlm. 494–503, Jun 2025, doi: 10.55606/jimek.v5i1.6399.

[3] D. D. Fitrianingsih, E. D. Apriliani, A. W. R. Ardianingrum, dan I. B. Igayanti, "IMPLEMENTASI DIGITALISASI REKAM MEDIS DALAM MENUNJANG PELAKSANAAN ELECTRONIC MEDICAL RECORD (EMR)," *Jurnal EduHealth Community*, vol. 3, no. 2, 2025, Diakses: 15 Agustus 2026. [Daring]. Tersedia pada: https://ejournal.abdiamanah.or.id/index.php/ec/article/download/79/74

[4] P. S. Samant, "Secure Cloud Services for the Healthcare Industry: Addressing Unique Challenges and Ensuring Compliance," *Int. J. Res. Appl. Sci. Eng. Technol.*, vol. 12, no. 4, hlm. 3095–3101, Apr 2024, doi: 10.22214/ijraset.2024.60636.

[5] A. Kohar dan H. P. Putro, "Ancaman Keamanan pada Sistem Informasi Manajemen Rumah Sakit," *Prosiding Seminar Nasional Informatika Medis (SNIMed)*, Des 2014, Diakses: 15 Agustus 2026. [Daring]. Tersedia pada: https://journal.uii.ac.id/snimed/article/download/6332/15

[6] C. Sorisa, C. L. Kiareni, dan J. Parhusip, "Etika Keamanan Siber: Studi Kasus Kebocoran Data BPJS Kesehatan di Indonesia," *Jurnal Sains Student Research*, vol. 2, no. 6, hlm. 586–593, Des 2024, doi: doi.org/10.61722/jssr.v2i6.2996.

[7] A. Andhavarapu, "Building Resilient Distributed Databases: Modern Approaches to High Availability," *International Journal of Research in Computer Application and Information Technology (IJRCAIT)*, vol. 8, no. 1, hlm. 1293–1307, Feb 2025, doi: 10.34218/ijrcait_08_01_096.

[8] Percona LLC, "Percona XtraBackup 8.0.35-33: Operations manual." Diakses: 16 Januari 2026. [Daring]. Tersedia pada: https://learn.percona.com/hubfs/Manuals/Percona_Xtra_Backup/Percona-XtraBackup-8.0/PerconaXtraBackup-8.0.35-33.pdf

[9] I. Sholihah dan C. Darujati, "Sistem Replikasi Basis Data berdasarkan MySQL menggunakan Container Docker," *Majalah Ilmiah Teknologi Elektro*, vol. 21, no. 2, Universitas Udayana, hlm. 209–218, 13 Desember 2022. doi: 10.24843/mite.2022.v21i02.p08.

[10] A. Kamiswara dan A. Wijaya, "Implementasi Master-Slave Sebagai Koneksi Baca dan Tulis Database Dengan Metode Replication," *Jurnal TEKNIKA*, vol. 19, no. 1, hlm. 47–58, Sep 2024.

[11] J. Susilo dan R. A. Mursalin, "Pengembangan Sistem Informasi Manajemen Rumah Sakit Menggunakan Framework PHP," *Jurnal Sains, Nalar, dan Aplikasi Teknologi Informasi*, vol. 2, no. 2, Sep 2023, doi: 10.20885/snati.v2i2.24.

[12] H. Susilo, D. Fauzi, M. Ihksan, dan N. Abdillah, "Sosialisasi Strategi Pemanfaatan Replikasi Basis Data (Master-Master dan Master-Slave) untuk Peningkatan Kinerja Aplikasi di RSIA Mutiara Bunda," *Jurnal Abdimas Saintika*, vol. 6, no. 1, hlm. 129–136, 2024, [Daring]. Tersedia pada: https://jurnal.syedzasaintika.ac.id

[13] I. S. Haryoto, G. Firmansyah, B. Tjahjono, A. M. Widodo, H. Akbar, dan N. S. Fatonah, "Evaluation and Optimization of MySQL Master-Slave Performance with Quantitative Methods on the Database of Psychology Test Applicants SIM PT XYZ at Polda Metro Jaya," *LOCUS JOURNAL: Research & Devotion*, vol. 4, no. 9, hlm. 8896–8905, Sep 2025.

[14] A. I. Ollong dan D. S. Kusumo, "Development of High Availability Database Infrastructure for OSS Projects with Monitoring Systems in Cloud Computing Environments," *International Journal on Information and Communication Technology (IJoICT)*, vol. 9, no. 2, hlm. 41–52, Des 2023, doi: 10.21108/ijoict.v9i2.761.

[15] N. K. Hulu, Tommy, dan D. Handoko, "Optimalisasi Kinerja CMS WordPress Melalui Penerapan Replikasi Database MySQL," *Jurnal Multidisiplin dan Sains*, vol. 1, no. 2, hlm. 160–172, Mar 2025, [Daring]. Tersedia pada: https://jurnal.compartdigital.com/index.php/judis

[16] H. T. Hidayat, N. Prihatin, M. Nasir, dan E. Astrid, "Design and Analysis of a Mobile Chatbot Application for Hospital Room Availability Using the Bryant Evaluation Method," *International Journal of Interactive Mobile Technologies (IJIM)*, vol. 20, no. 1, hlm. 34–51, Jan 2026, doi: 10.3991/ijim.v20i01.59507.

[17] E. D. Wuryanto dan S. mulyo Utomo, "IMPLEMENTASI SISTEM BACKUP DATABASE MYSQL OTOMATIS MENGGUNAKAN MYSQLDUMP DAN CRON JOB PADA INFRASTRUKTUR CLOUD," *JTIK (Jurnal Teknologi Informasi dan Komunikasi)*, vol. 17, no. 1, Mar 2026, Diakses: 15 Agustus 2026. [Daring]. Tersedia pada: https://ejurnal.provisi.ac.id/index.php/JTIKP/article/download/1786/1169

[18] A. Z. Abualkishik, A. A. Alwan, dan Y. Gulzar, "Disaster Recovery in Cloud Computing Systems: An Overview," *Int. J. Adv. Comput. Sci. Appl.*, vol. 11, no. 9, 2020, Diakses: 15 Agustus 2026. [Daring]. Tersedia pada: https://thesai.org/Downloads/Volume11No9/Paper_84-Disaster_Recovery_in_Cloud_Computing_Systems.pdf

[19] S. Abdul Moiz, S. P., V. G., dan S. N. Pal., "Database Replication: A Survey of Open Source and Commercial Tools," *Int. J. Comput. Appl.*, vol. 13, no. 6, Jan 2011, Diakses: 15 Agustus 2026. [Daring]. Tersedia pada: https://www.ijcaonline.org/volume13/number6/pxc3872469.pdf

[20] E. Cecchet, G. Candea, dan A. Ailamaki, "Middleware-based Database Replication: The Gaps Between Theory and Practice," *Proc. ACM SIGMOD Int'l Conf. on Management of Data*, Jun 2008, Diakses: 15 Agustus 2026. [Daring]. Tersedia pada: https://arxiv.org/pdf/0712.2773

[21] Ratih, N. Moniroh, dan F. Mahardika, "Strategi Keamanan VPS Menggunakan Pendekatan Berlapis: Studi Kasus Integrasi Cloudflare, 2FA, dan Monitoring," *Blend Sains Jurnal Teknik*, vol. 4, no. 2, hlm. 449–460, Nov 2025, doi: 10.56211/blendsains.v4i2.1315.

[22] I. G. J. Arissaputra, I. M. Sukarsa, P. W. Buana, dan N. W. Wisswani, "Binary Log Design for One-Way Data Replication with ZeroMQ," *International Journal of Modern Education and Computer Science*, vol. 10, no. 10, hlm. 22–30, Okt 2018, doi: 10.5815/ijmecs.2018.10.03.

[23] N. P. Florensia, M. P. Nurulita, G. Recita, dan J. Parhusip, "Peran Distribusi Nilai Rata-Rata dalam Menganalisis Kualitas Layanan IT dengan Pendekatan Statistika untuk Pengukuran Kinerja Sistem," *Jurnal Ilmu Teknik dan Informatika*, vol. 4, hlm. 30–38, 2024, doi: 10.51903/teknik.

[24] M. R. A. Arsandi dan A. Syaripudin, "Perancangan Sistem Monitoring Jaringan berbasis Web Server Terintegrasi Zabbix dan Notifikasi Telegram Pada PT Time Excelindo," *OKTAL : Jurnal Ilmu Komputer dan Science*, vol. 3, no. 6, Jun 2024, Diakses: 15 Agustus 2026. [Daring]. Tersedia pada: https://journal.mediapublikasi.id/index.php/oktal/article/download/3101/2793

[25] N. E. Lahagu, L. K. Simbolon, dan L. M. Sinaga, "Efektivitas Otomasi Administrasi Server Menggunakan Shell Script dan Cron Job pada Linux," *JURNAL ILMIAH KOMPUTER GRAFIS*, vol. 19, no. 1, hlm. 227–235, Jul 2026, Diakses: 15 Agustus 2026. [Daring]. Tersedia pada: https://journal.stekom.ac.id/index.php/pixel/article/download/3920/2498